



PUC Minas

Plataforma Node.js

Samuel Martins



PUC Minas

Introdução

Introdução - origem

- Até meados de 2008, apenas os browsers tinham interpretadores de Javascript:
- Motor responsável por interpretar os códigos escritos no browser;
- Páginas HTML, CSS e Javascript;
- Explica comportamentos de estilo diferentes para cada browser;
- O time de desenvolvimento do Google teve a ideia de expor o código fonte da V8 Engine, dando origem à plataforma Node.

Introdução - origem

- Possibilidade de unificar talentos;
- Desenvolvedores frontend tornaram-se fullstack quase que instantaneamente.
- Redução da curva de aprendizado;
- Redução de complexidade de escrita de aplicações
- Sistema single-thread padrão;
- Linguagem simplificada.

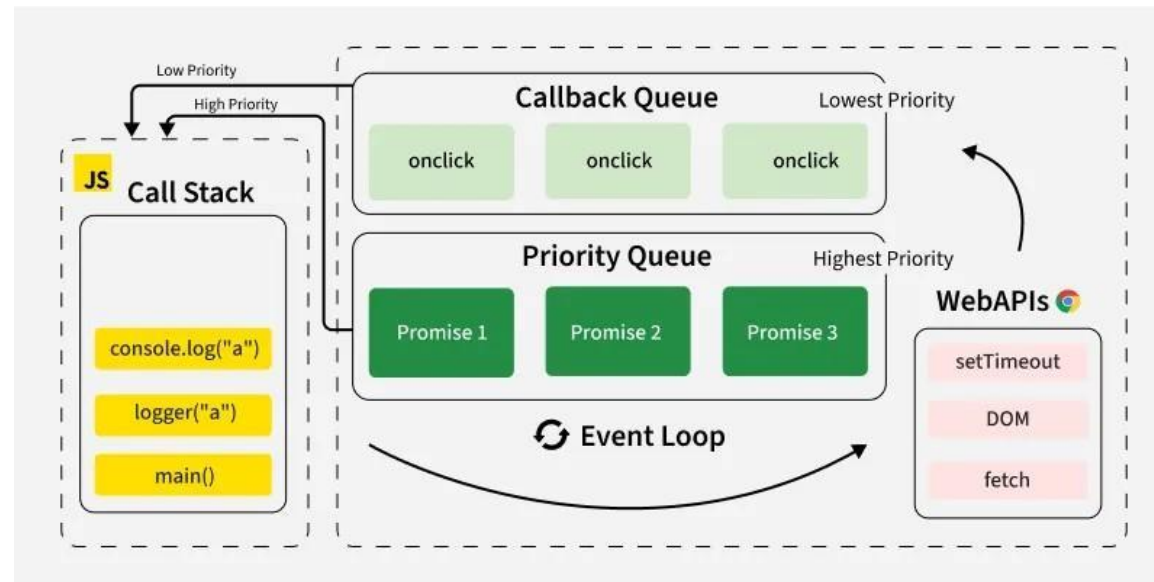


PUC Minas

Plataforma Node.js

Plataforma Node.js

- Possibilidade de execução de fluxos assíncronos mesmo em uma arquitetura Single thread, através de uma arquitetura baseada em eventos (Promises, setTimeouts, setInterval)



Fonte: <https://www.geeksforgeeks.org/javascript/what-is-an-event-loop-in-javascript/>

Introdução - origem

- Código fonte baseado em C++, exposto em 2009;
- Arquitetura single thread:
- Menor uso de memória e recursos;
- Menor complexidade;
- Fim dos deadlocks.



Fonte: <https://nodejs.org/en>



```
function a() {  
  console.log('function 1')  
}  
  
function b() {  
  function c() {  
    console.log('function 2')  
  }  
  
  c()  
  console.log('function 3')  
}  
  
a()  
b()  
  
// function 1  
// function 2  
// function 3
```

Fonte: próprio autor



```
1  function execAsync(frase) {  
2      setTimeout(function() {  
3          console.log(frase);  
4      }, 3000)  
5  }  
6  
7  function funcao1() {  
8      console.log('funcao 1');  
9  }  
10  
11  execAsync('Execução async');  
12  funcao1()
```

Fonte: próprio autor



```
1  function execAsync(frase) {  
2      setTimeout(function() {  
3          console.log(frase);  
4      })  
5  }  
6  
7  function funcao1() { console.log('funcao 1'); }  
8  function funcao2() { console.log('funcao 2'); }  
9  function funcao3() { console.log('funcao 3'); }  
10  
11  execAsync('Execução async');  
12  funcao1()  
13  funcao2()  
14  funcao3()
```

Fonte: próprio autor

Qual o resultado do código anterior?

- a) Execução async, funcao 1, funcao 2, funcao 3;
- b) funcao 1, Execução async, funcao 2, funcao 3;
- c) funcao 1, funcao 2, funcao 3, Execução async;
- d) Nenhuma das opções. Erro ao executar as funções.

Qual o resultado do código anterior?

- a) Execução async, funcao 1, funcao 2, funcao 3;
- b) funcao 1, Execução async, funcao 2, funcao 3;
- c) funcao 1, funcao 2, funcao 3, Execução async;**
- d) Nenhuma das opções. Erro ao executar as funções.



PUC Minas

Javascript e Typescript

Javascript

- Linguagem de programação interpretada (no caso, pela engine do browser);
- Utilizada para escrever aplicações na plataforma Node.js;
- Linguagem fracamente tipada: possível declarar uma variável que pode receber qualquer tipo a qualquer momento;
- Adiciona interatividade nas aplicações web.

Javascript

- Possui diversas especificações:
 - EcmaScript 1, 2, 3....7, 8;
- As especificações são definidas por um conjunto de especialistas da comunidade open-source (Google, Mozilla, Microsoft...) e definem as funcionalidades presentes no Javascript;

Typescript

- Linguagem de programação que estende os recursos originais do JavaScript;
- Adiciona tipagem estática opcional e inúmeras outras funcionalidades que trazem mais robustez à nossa aplicação;
- Compatível com todos os recursos mais recentes do javascript;
- Necessita um compilador que traduz o Typescript para Javascript, uma vez que o node trabalha exclusivamente com Javascript.



PUC Minas

Sistema de Módulos

Sistema de Módulos

- Componentização;
- Em Node, TUDO pode virar um módulo:
- String, number, object, functions, classes...;

```
● ● ●  
  
// function  
module.exports = function(x, y) {  
    return x + y  
}
```

Fonte: próprio autor

Import com require



```
// import module  
const sum = require('./sum.js')  
  
const result = sum(1, 2)
```

Fonte: próprio autor

Import (syntaxe mais recente)



```
export default function (x, y) {  
  return x + y  
}
```



```
import sum from './sum.js'  
  
const result = sum(1, 2)
```

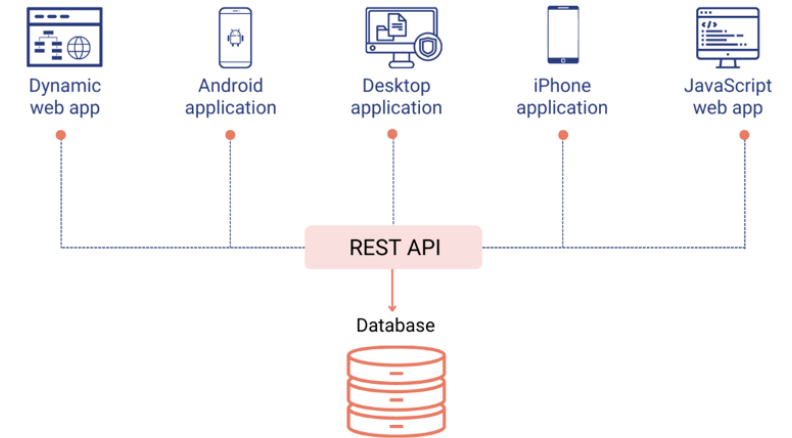


PUC Minas

Implementação de APIs

Implementação de APIs

- API: Application Programming Interface;
- No contexto de API, aplicação refere-se a qualquer software;
- **Interface:** contrato definido entre duas aplicações. Esse contrato define a forma como as aplicações se comunicam



Fonte: <https://datasciencedojo.com/blog/understanding-rest-api/>

Implementação de APIs

- No caso de uma API REST, a comunicação se dá através de chamadas HTTP;
- O cliente envia uma solicitação ao servidor, através de um verbo HTTP;
- O servidor processa essa solicitação e devolve uma resposta ao cliente.

Implementação de APIs

- O Node.js oferece um conjunto de métodos para implementação de APIs em HTTP de forma nativa, ou através de bibliotecas prontas, como:
 - **Express:** biblioteca que configura um servidor HTTP juntamente com a possibilidade de expor uma API REST;
 - **Nestjs:** framework que abstrai o Express criando dando a possibilidade de implementarmos arquiteturas robustas e escaláveis, como microserviços, modularização e etc.



PUC Minas

Introdução ao Nest.js

Introdução ao NestJs

- NestJS é um framework para construção de aplicativos Node.js escaláveis e eficientes, baseado no TypeScript.
- Ele utiliza os princípios da arquitetura modular muito inspirada no Angular e orientada a objetos para fornecer uma estrutura robusta para o desenvolvimento de APIs e aplicativos web;
- O NestJs atua como um framework que encapsula os recursos de servidor providos pelo Express (ou o fastify) para fazer com que seja possível estruturar a aplicação de uma forma muito mais robusta e escalável.

Introdução ao NestJs - Services

- São classes responsáveis por conter a lógica de negócio da aplicação, como a interface com o banco, integrações com APIs externas e quaisquer outras regras pertinentes.
- Podem ser injetados em controladores, módulos ou outros provedores de serviço.

```
// products.service.ts
import { Injectable } from
 '@nestjs/common';

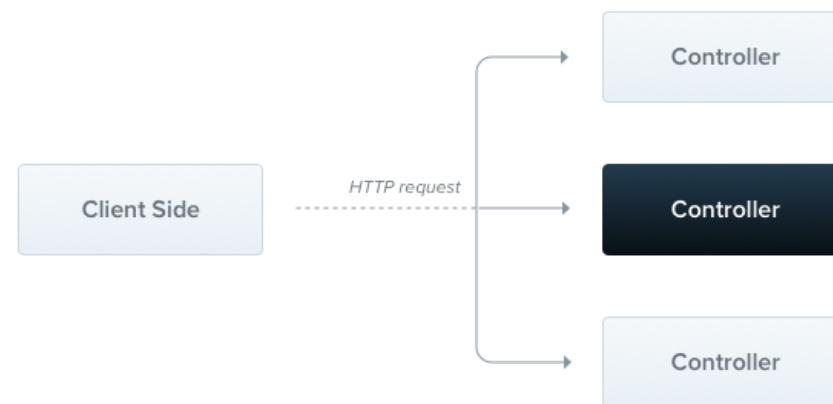
@Injectable()
export class ProductService {
  findAll(): string {
    return 'Lista de produtos';
  }
}
```

Introdução ao NestJs

- O conceito de injeção de dependências no Nest (ou dependency injection como na documentação) facilita muito o reuso de código e a visualização de dependências entre partes da aplicação;
- Uma classe que atua como um Provider pode ser injetada em qualquer construtor, desde que esteja dentro de um mesmo módulo;
- Uma classe injetável (Injectable) possui uma instância única na aplicação, atuando como um singleton.

Introdução ao NestJs - Controllers

- São responsáveis por lidar com as requisições HTTP e retornar as respostas adequadas;
- Decorados com metadados que definem os endpoints da API e os métodos HTTP que cada endpoint suporta.



Fonte: <https://docs.nestjs.com/controllers>

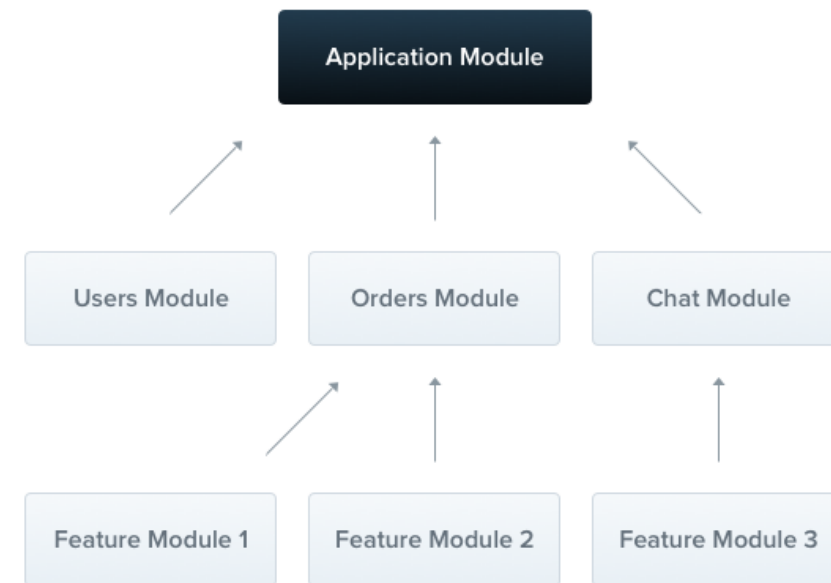
```
// products.controller.ts
import { Controller, Get } from '@nestjs/common';
import { ProductService } from '../products.service';

@Controller('products')
export class ProductsController {
  constructor(private readonly productService: ProductService) {}

  @Get()
  findAll(): string {
    return this.productService.findAll();
  }
}
```

Introdução ao NestJs - Módulos

- Segundo o NestJs, muitos frameworks e bibliotecas têm o propósito de resolver problemas isolados, mas a grande maioria falhou em resolver um problema arquitetural;
- Arquitetura 100% modular:
- Utiliza módulos para organizar e estruturar a aplicação de forma modular e escalável.



Fonte: <https://docs.nestjs.com/modules>



PUC Minas